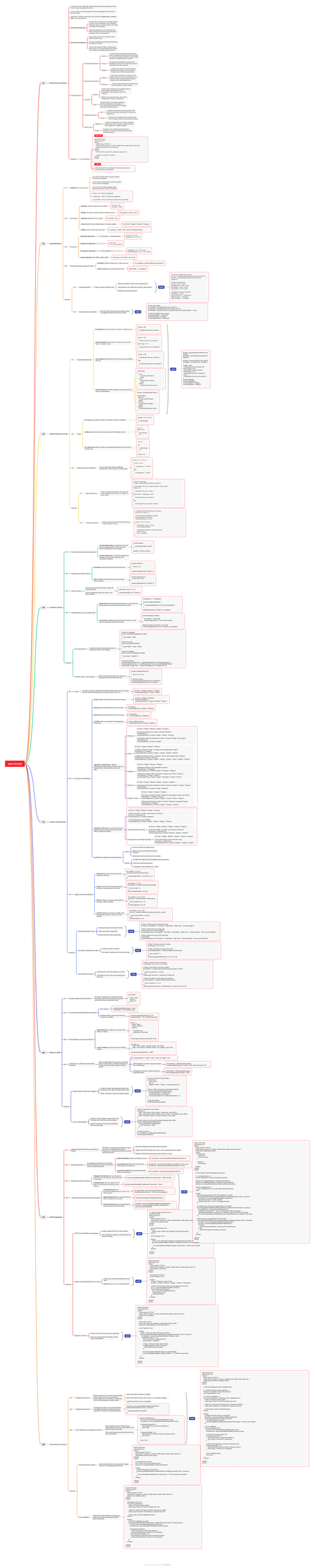

Java Script

#1 Introduction to JavaScript

- JavaScript (JS) is a high-level, interpreted programming language primarily used to make web pages interactive.
- It is one of the core technologies of the web, alongside HTML (structure) and CSS (style).
- JavaScript is versatile and can be used on both the **client-side** and **server-side** of web development.
- **Client-side JavaScript**
  - Runs directly in the browser. It enables dynamic content, such as animations, form validations, interactive maps, and other elements that make web pages more engaging.
  - Client-side JavaScript is executed after the HTML and CSS have been loaded and helps in creating a richer user experience.
- **Server-side JavaScript**
  - Runs on the server, most commonly using platforms like Node.js.
  - It handles tasks like interacting with databases, file systems, and APIs.
  - Server-side JavaScript helps create dynamic web content, such as generating HTML based on user data or handling backend services.
- Tools & Libraries
  - Front end Frameworks
    - React – A JavaScript library developed by Facebook, used for building interactive UIs. React focuses on component-based architecture and is highly efficient in rendering dynamic content.
    - Vue.js – A progressive framework for building user interfaces, known for its simplicity and ease of integration with other projects.
    - Angular – A TypeScript-based front-end framework developed by Google, used for building complex, large-scale web applications.
  - Back end Frameworks
    - Node.js – A JavaScript runtime built on Chrome's V8 engine. It allows you to run JavaScript on the server side and is widely used for building scalable, event-driven applications.
    - Express.js – A minimal, fast framework for building web servers and APIs on top of Node.js.
  - Libraries
    - jQuery
      - A fast, small, and feature-rich JavaScript library that simplifies tasks like HTML DOM manipulation, event handling, and AJAX requests.
      - While it's less popular today with modern frameworks, it remains foundational.
    - D3.js – A powerful library for data visualization, allowing developers to bind data to a Document Object Model (DOM) and apply data-driven transformations.
  - Testing Frameworks
    - Jest – A delightful JavaScript testing framework with a focus on simplicity and support for large web applications.
    - Mocha – A feature-rich JavaScript test framework that runs on Node.js, useful for asynchronous testing.
  - Build Tools
    - Webpack – A static module bundler for modern JavaScript applications, which compiles multiple modules into a single file or smaller bundles.
    - Parcel – A simpler, zero-config bundler that allows developers to package their JavaScript projects with minimal setup.
- Handson – VS Code Setup
  - index.html

    ```
    <!DOCTYPE html>
    <html lang="en">
    <head>
      <meta charset="UTF-8">
      <meta name="viewport" content="width=device-width, initial-scale=1.0">
      <title>JavaScript Console Log</title>
    </head>
    <body>
      <h1>Check the console for JavaScript output</h1>

      <script src="script.js"></script>
    </body>
    </html>
    ```

  - script.js

    ```
    // This will print "Hello, JavaScript!" in the browser console
    console.log("Hello, JavaScript!");
    ```

#2 JavaScript Basics

- #1 **Variables:** let, const, and var
  - let: Used to declare block-scoped variables that can be reassigned.
  - const: Used to declare block-scoped variables that cannot be reassigned.
  - var: A function-scoped variable (older, generally avoided in modern JavaScript).

    ```
    let age = 25; // Can be reassigned

    const name = "John"; // Cannot be reassigned

    var isStudent = true; // Function-scoped, rarely used today
    ```

- #2 Data Types
  - **Numbers:** Integers, floating-point numbers.

    ```
    let num = 10;
    let price = 19.99;
    ```

  - **Strings:** Text data, enclosed in single or double quotes. – `let greeting = "Hello, world!";`
  - **Booleans:** Represents true or false. – `let isAdult = true;`
  - **Arrays:** Used to store multiple values in a single variable. – `let fruits = ["Apple", "Banana", "Orange"];`
  - **Objects:** Used to store key-value pairs – `let person = { name: "John", age: 30, isStudent: false };`
- #3 Operators
  - **Arithmetic Operators:** +, -, *, /, % (modulus), ** (exponentiation)

    ```
    let sum = 5 + 3; // 8
    let product = 5 * 2; // 10
    ```

  - **Assignment Operators:** =, +=, -=, *=, /=

    ```
    let x = 10;
    x += 5; // x is now 15
    ```

  - **Comparison Operators:** ==, === (strict equality), !=, !==, <, >, <=, >=

    ```
    let isEqual = (5 == "5"); // true
    let isStrictEqual = (5 === "5"); // false
    ```

  - **Logical Operators:** && (AND), || (OR), ! (NOT) – `let result = (5 > 3 && 2 < 4); // true`
- #4 Basic Input/Output: prompt and alert
  - **prompt():** Displays a dialog box to take user input. – `let userName = prompt("What is your name?");`
  - **alert():** Displays a pop-up alert to the user. – `alert("Hello, " + userName);`
- Exercise
  - #1 Simple Calculator – Create a simple calculator that:
    - Takes two numbers from the user using prompt.
    - Calculates the sum, difference, product, and quotient.
    - Displays the results using alert.
    - Code

      ```
      // Get two numbers from the user
      let num1 = parseFloat(prompt("Enter the first number:"));
      let num2 = parseFloat(prompt("Enter the second number:"));

      // Perform calculations
      let sum = num1 + num2;
      let difference = num1 - num2;
      let product = num1 * num2;
      let quotient = num1 / num2;

      // Display the results using alert
      alert("Sum: " + sum);
      alert("Difference: " + difference);
      alert("Product: " + product);
      alert("Quotient: " + quotient);
      ```

  - #2 Store and Print User Details – Store the user's name, age, and student status in variables and print them to the console.
    - Code

      ```
      // Store user details
      let userName = prompt("What is your name?");
      let userAge = parseInt(prompt("How old are you?"));
      let isStudent = prompt("Are you a student? (yes/no)").toLowerCase() === "yes";

      // Print user details to the console
      console.log("Name", userName);
      console.log("Age", userAge);
      console.log("Student:", isStudent);
      ```

#3 Control Structures & Loops

- #1 Conditional Statements
  - **if statement:** Executes a block of code if a condition is true.

    ```
    if (age >= 18)
    {
       console.log("You are an adult.");
    }
    ```

  - **else if statement:** Adds another condition to check if the initial if condition is false.

    ```
    if (age >= 18)
    {
       console.log("You are an adult.");
    }
    else if (age >= 13)
    {
       console.log("You are a teenager.");
    }
    ```

  - **else statement:** Executes if none of the previous conditions are true.

    ```
    if (age >= 18)
    {
       console.log("You are an adult.");
    }
    else
    {
       console.log("You are not an adult.");
    }
    ```

  - **switch statement:** Used for multiple choices based on the value of an expression.

    ```
    switch (day)
    {
      case 1:
        console.log("Monday");
        break;
      case 2:
        console.log("Tuesday");
        break;
      default:
        console.log("Unknown day");
    }
    ```

    ```
    let day = prompt("Enter Char?");

    switch (day) {
    case "A":
      console.log("Monday");
      break;
    case "B":
      console.log("Tuesday");
      break;
    default:
      console.log("Unknown day");
    }
    ```

  - code

    ```
    let age = parseInt(prompt("What is your Age?"));
    let height = parseFloat(prompt("enter ur height:"));

    let age2 = prompt("What is your Age?");
    let height2 = prompt("enter ur height:");

    if (age == 18) {
     console.log("You are exactly 18");
    } else if (age >= 18) {
     console.log("You are a adult.");
    } else if (age >= 13) {
     console.log("You are a Teenager.");
    } else {
     console.log("You are not an adult.");
    }
    console.log(age);
    console.log(height);
    console.log(age + height);
    console.log(age2 + height2);
    ```

- #2 Loops
  - **for loop:** Runs a block of code for a specific number of iterations.

    ```
    for (let i = 0; i < 5; i++)
    {
       console.log(i);
    }
    ```

  - **while loop:** Runs a block of code as long as the condition is true.

    ```
    let i = 0;
    while (i < 5)
    {
       console.log(i);
       i++;
    }
    ```

  - **do...while loop:** Similar to while, but guarantees the block of code will run at least once.

    ```
    let i = 0;

    do
    {
       console.log(i);
       i++;
    }

    while (i < 5);
    ```

- #3 Nesting Loops and Conditions – You can nest loops inside conditional statements or other loops for complex logic.

  ```
  for (let i = 1; i <= 3; i++)
  {
    if (i % 2 === 0)
    {
      console.log(i + " is even");
    }
    else
    {
      console.log(i + " is odd");
    }
  }
  ```

- Exercise
  - #1 Age Classification – Create a program that takes a user's age and prints whether they are a minor, an adult, or a senior citizen.

    ```
    // Get the user's age
    let age = parseInt(prompt("Enter your age:"));

    // Determine if the user is a minor, adult, or senior citizen
    if (age < 18)
    {
      console.log("You are a minor.");
    }
    else if (age >= 18 && age <= 60)
    {
      console.log("You are an adult.");
    }
    else
    {
      console.log("You are a senior citizen.");
    }
    ```

  - #2 Fibonacci Sequence – Write a program to print the first 10 Fibonacci numbers using a for loop.

    ```
    // Initialize the first two Fibonacci numbers
    let num1 = 0, num2 = 1;

    // Print the first 10 Fibonacci numbers
    console.log(num1); // Print 0
    console.log(num2); // Print 1

    for (let i = 3; i <= 10; i++)
    {
      let nextNum = num1 + num2;
      console.log(nextNum);

      // Update the values for the next iteration
      num1 = num2;
      num2 = nextNum;
    }
    ```

#4 Functions & Scope

- #1 Function Declarations and Expressions
  - **Function Declaration:** A named function that can be called anywhere in the code, even before its definition, due to hoisting
  - **Function Expression:** A function assigned to a variable. It can only be called after the expression is defined

    ```
    function greet()
    {
       console.log("Hello, world!");
    }

    greet(); // Call the function
    ```

- #2 Parameters and Return Values
  - **Parameters:** Input values passed into the function.

    ```
    function add(a, b)
    {
       return a + b;
    }

    console.log(add(3, 5)); // Output: 8
    ```

  - **Return Values:** The result that a function returns after execution.

    ```
    function square(num) {
       return num * num;
    }

    console.log(square(4)); // Output: 16
    ```

- #3 Arrow Functions
  - Arrow functions provide a shorter syntax for writing functions.
  - They are especially useful for concise, one-liner functions.

    ```
    const add = (a, b) => a + b;

    console.log(add(2, 3)); // Output: 5
    ```

- #4 Understanding Local vs. Global Scope
  - **Global Scope:** Variables declared outside of any function are global and can be accessed anywhere in the script.

    ```
    let globalVar = "I am global";

    function displayGlobalVar()
    {
       console.log(globalVar); // Can access globalVar
    }

    displayGlobalVar(); // Output: "I am global"
    ```

  - **Local Scope:** Variables declared inside a function are local and only accessible within that function.

    ```
    function displayLocalVar()
    {
       let localVar = "I am local";
       console.log(localVar); // Can access localVar
    }

    displayLocalVar(); // Output: "I am local"
    // console.log(localVar); // Error: localVar is not defined
    ```

- Exercise
  - Area Calculators – Create functions to calculate the area of a rectangle, a circle, and a triangle.

    ```
    // Area of a rectangle
    function areaOfRectangle(length, width)
    {
       return length * width;
    }

    // Area of a circle
    function areaOfCircle(radius)
    {
       return Math.PI * radius * radius;
    }

    // Area of a triangle
    function areaOfTriangle(base, height)
    {
       return (base * height) / 2;
    }

    // Test the functions
    console.log("Rectangle Area: " + areaOfRectangle(5, 3)); // Rectangle Area: 15
    console.log("Circle Area: " + areaOfCircle(4)); // Circle Area: 50.26548245743669
    console.log("Triangle Area: " + areaOfTriangle(5, 4)); // Triangle Area: 10
    ```

  - Find the Larger of Two Numbers – Write a function that takes two numbers as arguments and returns the larger one.

    ```
    function largerNumber(a, b)
    {
       return a > b ? a : b;
    }

    // Test the function
    console.log(largerNumber(10, 5)); // Output: 10
    console.log(largerNumber(7, 7)); // Output: 7
    ```

#5 Arrays & Array Methods

- #1 Arrays – An array is a special variable that can hold multiple values at once. Arrays are zero-indexed, meaning the first element has an index of 0

  ```
  let fruits = ["Apple", "Banana", "Mango"];
  console.log(fruits[0]); // Output: "Apple"
  ```

- #2 Common Array Methods
  - **push():** Adds a new element to the end of an array.

    ```
    let fruits = ["Apple", "Banana"];
    fruits.push("Mango");
    console.log(fruits); // ["Apple", "Banana", "Mango"]
    ```

  - **pop():** Removes the last element from an array.

    ```
    fruits.pop();
    console.log(fruits); // ["Apple", "Banana"]
    ```

  - **shift():** Removes the first element from an array.

    ```
    fruits.shift();
    console.log(fruits); // ["Banana"]
    ```

  - **unshift():** Adds a new element to the beginning of an array.

    ```
    fruits.unshift("Orange");
    console.log(fruits); // ["Orange", "Banana"]
    ```

  - **splice(start, deleteCount, ...items):** Adds/removes items to/from an array. Can add new elements, remove existing ones, or do both.
    - Remove

      ```
      let fruits = ["Apple", "Banana", "Mango", "Orange"];

      // Removing 1 element from index 1 (removes "Banana")
      fruits.splice(1, 1);
      console.log(fruits); // Output: ["Apple", "Mango", "Orange"]

      // Removing 2 elements starting from index 1 (removes "Mango" and "Orange")
      fruits.splice(1, 2);
      console.log(fruits); // Output: ["Apple"]
      ```

    - Add

      ```
      let fruits = ["Apple", "Banana", "Mango"];

      // Adding 1 element at index 1 ("Orange" will be added after "Apple")
      fruits.splice(1, 0, "Orange");
      console.log(fruits); // Output: ["Apple", "Orange", "Banana", "Mango"]

      // Adding 2 elements at index 2 ("Grapes", "Peach" will be added after "Banana")
      fruits.splice(2, 0, "Grapes", "Peach");
      console.log(fruits); // Output: ["Apple", "Orange", "Grapes", "Peach", "Banana", "Mango"]
      ```

    - Replace

      ```
      let fruits = ["Apple", "Banana", "Mango"];

      // Replacing "Banana" with "Pineapple" at index 1
      fruits.splice(1, 1, "Pineapple");
      console.log(fruits); // Output: ["Apple", "Pineapple", "Mango"]

      // Replacing "Mango" with two elements: "Grapes" and "Peach" at index 2
      fruits.splice(2, 1, "Grapes", "Peach");
      console.log(fruits); // Output: ["Apple", "Pineapple", "Grapes", "Peach"]
      ```

    - Remove

      ```
      let fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"];

      // Removing all elements starting from index 2 (removes "Mango", "Orange", "Grapes")
      fruits.splice(2);
      console.log(fruits); // Output: ["Apple", "Banana"]
      ```

    - Remove + Add

      ```
      let fruits = ["Apple", "Banana", "Mango"];

      // Removing 1 element at index 1 ("Banana") and adding "Orange" and "Peach"
      fruits.splice(1, 1, "Orange", "Peach");
      console.log(fruits); // Output: ["Apple", "Orange", "Peach", "Mango"]

      // Removing 2 elements at index 2 ("Peach", "Mango") and adding "Grapes"
      fruits.splice(2, 2, "Grapes");
      console.log(fruits); // Output: ["Apple", "Orange", "Grapes"]
      ```

  - **slice(start, end):** Returns a new array from the original array, extracting elements from start index to end (not inclusive).

    ```
    let fruits = ["Apple", "Banana", "Mango", "Orange"];

    // Slice from index 1 to index 2 (not inclusive of index 2)
    let citrus = fruits.slice(1, 2);
    console.log(citrus); // Output: ["Banana"]

    // The original array is not modified
    console.log(fruits); // Output: ["Apple", "Banana", "Mango", "Orange"]
    ```

    - Extracting portion of array

      ```
      let fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"];

      // Slice from index 1 to index 4 (not inclusive of index 4)
      let portion = fruits.slice(1, 4);
      console.log(portion); // Output: ["Banana", "Mango", "Orange"]
      ```

    - Slicing From a Start Index to the End

      ```
      let fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"];

      // Slice from index 2 to the end of the array
      let portion = fruits.slice(2);
      console.log(portion); // Output: ["Mango", "Orange", "Grapes"]
      ```

  - Key Differences Between slice() and splice()
    - slice()
      - Does not modify the original array.
      - Returns a new array with the extracted elements.
      - Parameters: (start, end) (end is not included).
    - splice()
      - Modifies the original array by adding/removing elements.
      - Returns the removed elements.
      - Parameters: (start, deleteCount, ...items).
- #3 Higher-Order Array Methods
  - **forEach():** Executes a function for each element in the array.

    ```
    let numbers = [1, 2, 3];
    numbers.forEach(function(number)
    {
       console.log(number * 2); // Prints 2, 4, 6
    });
    ```

  - **map():** Creates a new array by applying a function to each element in the array.

    ```
    let numbers = [1, 2, 3];
    let doubled = numbers.map(function(number)
    {
       return number * 2;
    });
    console.log(doubled); // [2, 4, 6]
    ```

  - **filter():** Creates a new array with elements that pass a given condition.

    ```
    let numbers = [1, 2, 3, 4, 5];
    let evens = numbers.filter(function(number)
    {
       return number % 2 === 0;
    });
    console.log(evens); // [2, 4]
    ```

  - **reduce():** Reduces an array to a single value by executing a reducer function on each element.

    ```
    let numbers = [1, 2, 3, 4];
    let sum = numbers.reduce(function(accumulator, current)
    {
       return accumulator + current;
    }, 0);
    console.log(sum); // 10
    ```

- Exercise
  - Manipulating Movies Array
    - Create an array of five favorite movies.
    - Add a new movie using push().
    - Remove the first movie using shift().
    - code

      ```
      // Step 1: Create an array of favorite movies
      let movies = ["Inception", "The Matrix", "Interstellar", "Fight Club", "The Dark Knight"];

      // Step 2: Add a new movie using push
      movies.push("The Lord of the Rings");
      console.log(movies); // ["Inception", "The Matrix", "Interstellar", "Fight Club", "The Dark Knight", "The Lord of the Rings"]

      // Step 3: Remove the first movie using shift
      movies.shift();
      console.log(movies); // ["The Matrix", "Interstellar", "Fight Club", "The Dark Knight", "The Lord of the Rings"]
      ```

  - Use map() to Double Each Value
    - Create an array of numbers.
    - Use map() to double each value in the array.
    - code

      ```
      // Step 1: Create an array of numbers
      let numbers = [1, 2, 3, 4, 5];

      // Step 2: Use map to double each value
      let doubledNumbers = numbers.map(function(number)
      {
         return number * 2;
      });
      console.log(doubledNumbers); // [2, 4, 6, 8, 10]
      ```

  - Use reduce() and filter()
    - Use reduce() to sum the numbers in an array.
    - Use filter() to return only even numbers from the array.
    - code

      ```
      // Step 1: Create an array of numbers
      let numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

      // Step 2: Use reduce to sum the numbers
      let totalSum = numbers.reduce(function(accumulator, current)
      {
         return accumulator + current;
      }, 0);
      console.log("Total Sum:", totalSum); // Output: 55

      // Step 3: Use filter to return only even numbers
      let evenNumbers = numbers.filter(function(number)
      {
         return number % 2 === 0;
      });
      console.log("Even Numbers:", evenNumbers); // Output: [2, 4, 6, 8, 10]
      ```

#6 Objects & JSON

- #1 Creating Objects (Key-Value Pairs) – An object in JavaScript is a collection of key-value pairs. Keys are also known as properties, and each key has an associated value.

  ```
  let student =
  {
     name: "John",
     age: 20,
     grade: "A"
  };
  ```

- #2 Accessing and Modifying Object Properties
  - Dot notation

    ```
    console.log(student.name); // "John"
    student.age = 21; // Modify the age
    ```

  - Bracket notation (useful when the key is stored as a variable):

    ```
    console.log(student["grade"]); // "A"
    student["grade"] = "B"; // Modify the grade
    ```

- #3 Nested Objects and Arrays of Objects
  - **Nested Objects:** An object can contain other objects as values.

    ```
    let car = {
       brand: "Tesla",
       model: "Model S",
       features:
       {
          autopilot: true,
          batteryLife: "500 miles"
       }
    };

    console.log(car.features.autopilot); // true
    ```

  - **Array of Objects:** An array can store multiple objects.

    ```
    let books = [
       { title: "1984", author: "George Orwell", year: 1949 },
       { title: "The Catcher in the Rye", author: "J.D. Salinger", year: 1951 }
    ];

    console.log(books[0].title); // "1984"
    ```

- #4 Introduction to JSON (JavaScript Object Notation) – JSON is a lightweight format used to represent data. It looks similar to JavaScript objects, but the keys must be strings wrapped in double quotes.
  - `let studentJSON = '{"name": "John", "age": 20, "grade": "A"}';`
  - JSON.stringify(): Converts a JavaScript object into a JSON string.

    ```
    let jsonString = JSON.stringify(student);
    console.log(jsonString); // '{"name":"John","age":20,"grade":"A"}'
    ```

  - JSON.parse(): Converts a JSON string into a JavaScript object.

    ```
    let jsonObject = JSON.parse(jsonString);
    console.log(jsonObject.name); // "John"
    ```

- Exercise
  - Student Object with Array of Subjects
    - Create an object representing a student with properties: name, age, and subjects (an array).
    - Write a function to print the student's details.
    - code

      ```
      // Step 1: Create the student object
      let student = {
         name: "Alice",
         age: 22,
         subjects: ["Math", "Physics", "Computer Science"]
      };

      // Step 2: Write a function to print student details
      function printStudentDetails(student) {
         console.log("Name:", student.name);
         console.log("Age:", student.age);
         console.log("Subjects:", student.subjects.join(", "));
      }

      // Test the function
      printStudentDetails(student);
      ```

  - Array of Book Objects
    - Create an array of objects representing a list of books, each with title, author, and year.
    - Write a function to find all books published after the year 2000.
    - code

      ```
      // Step 1: Create the array of books
      let books = [
         { title: "The Da Vinci Code", author: "Dan Brown", year: 2003 },
         { title: "Harry Potter and the Goblet of Fire", author: "J.K. Rowling", year: 2000 },
         { title: "The Road", author: "Cormac McCarthy", year: 2006 }
      ];

      // Step 2: Write a function to find books published after 2000
      function findBooksAfter2000(books) {
         return books.filter(function(book) {
            return book.year > 2000;
         });
      }

      // Test the function
      let recentBooks = findBooksAfter2000(books);
      console.log(recentBooks);
      ```

#7 DOM Manipulation

- #1 Understanding the DOM (Document Object Model) – The DOM is a programming interface for web documents. It represents the page so programs can change the document structure, style, and content.
  - Document: Represents the entire HTML document.
  - Object: Each HTML element (e.g., <div>, <p>) is represented as an object.
  - Model: A tree-like structure where each element is a node.
- #2 Selecting Elements – To manipulate the DOM, we first need to select the elements we want to work with. Common methods include:
  - **getElementById():** Selects an element by its id. – `let element = document.getElementById("myElement");`
  - **querySelector():** Selects the first element that matches a CSS selector.

    ```
    let element = document.querySelector(".myClass"); // For class
    let element = document.querySelector("#myId"); // For ID
    ```

  - **querySelectorAll():** Selects all elements that match a CSS selector. – `let elements = document.querySelectorAll("p");`
- #3 Changing Content and Styles
  - **Changing Text Content:** We can change the content of an HTML element using textContent or innerHTML. – `document.getElementById("myElement").textContent = "Hello, World!";`
  - **Changing Styles:** We can change the CSS styles of an element by modifying its style property. – `document.getElementById("myElement").style.color = "blue";`
- #4 Adding and Removing Elements Dynamically
  - **Creating Elements:** Use document.createElement() to create a new HTML element.

    ```
    let newElement = document.createElement("p");
    newElement.textContent = "This is a new paragraph.";
    ```

  - **Appending Elements:** Use appendChild() to add a new element to the DOM. – `document.body.appendChild(newElement);`
  - **Removing Elements:** Use removeChild() to remove an element from the DOM.

    ```
    let parent = document.getElementById("parentElement");
    let child = document.getElementById("childElement");
    parent.removeChild(child);
    ```

- code

  ```
  <!DOCTYPE html>
  <html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>DOM Manipulation</title>
    <style>
      #myElement {
        color: red;
        font-size: 20px;
      }

      .newClass {
        color: green;
      }
    </style>
  </head>
  <body>

    <h1 id="header">DOM Manipulation Demo</h1>

    <!-- Existing Elements -->
    <div id="myElement">This is the original text!</div>

    <button id="changeTextButton">Change Text</button>
    <button id="addElementButton">Add New Element</button>
    <button id="removeElementButton">Remove Element</button>

    <div id="parentElement">
      <p id="childElement">I am a removable child element.</p>
    </div>

    <script>
      // Selecting an element by its ID and changing its content
      document.getElementById("changeTextButton").addEventListener("click", function() {
        let element = document.getElementById("myElement");
        element.textContent = "Hello, World!";
        element.style.color = "blue"; // Changing the style
      });

      // Creating a new element and appending it to the DOM
      document.getElementById("addElementButton").addEventListener("click", function() {
        let newElement = document.createElement("p");
        newElement.textContent = "This is a dynamically added paragraph!";
        newElement.className = "newClass"; // Adding a class to style the new element
        document.body.appendChild(newElement); // Appending the new element to the body
      });

      // Removing an existing element from the DOM
      document.getElementById("removeElementButton").addEventListener("click", function() {
        let parent = document.getElementById("parentElement");
        let child = document.getElementById("childElement");
        if (child) {
          parent.removeChild(child); // Removing the child element
        } else {
          alert("Child element is already removed!");
        }
      });
    </script>

  </body>
  </html>
  ```

- Exercise
  - HTML Form with Button and Message
    - Create a simple HTML form with a button.
    - Use JavaScript to display a message when the button is clicked.
    - code

      ```
      <!-- Step 1: Create an HTML form -->
      <!DOCTYPE html>
      <html lang="en">
      <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=
        <title>Form Example</title>
      </head>
      <body>

        <h1>Click the Button</h1>
        <form>
          <button type="button" id="myButton">Click Me!</button>
        </form>

        <p id="message"></p>

        <script>
          // Step 2: Use JavaScript to display a message when the button is c
          document.getElementById("myButton").addEventListener("click", function()
          {
            document.getElementById("message").textContent = "Button was clicked!";
          });
        </script>

      </body>
      </html>
      ```

  - Create a List Dynamically from an Array
    - Create a list of items dynamically from an array.
    - Display the list in a <ul> using JavaScript.
    - code

      ```
      <!DOCTYPE html>
      <html lang="en">
      <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Dynamic List</title>
      </head>
      <body>

        <h1>Favorite Fruits</h1>
        <ul id="fruitList"></ul>

        <script>
          // Step 1: Create an array of fruits
          let fruits = ["Apple", "Banana", "Mango", "Orange", "Pineapple"];

          // Step 2: Dynamically create a list and display it in the <ul>
          let ul = document.getElementById("fruitList");
          fruits.forEach(function(fruit) {
            let li = document.createElement("li");
            li.textContent = fruit;
            ul.appendChild(li);
          });
        </script>

      </body>
      </html>
      ```

  - Dynamic To-Do List
    - Create a to-do list where users can add tasks.
    - Users can remove tasks by clicking on them using removeChild().
    - code

      ```
      <!DOCTYPE html>
      <html lang="en">
      <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>To-Do List</title>
      </head>
      <body>

        <h1>To-Do List</h1>
        <input type="text" id="taskInput" placeholder="Enter a task">
        <button id="addTaskButton">Add Task</button>

        <ul id="taskList"></ul>

        <script>
          // Step 1: Add a task when the button is clicked
          document.getElementById("addTaskButton").addEventListener("click", function() {
            let taskInput = document.getElementById("taskInput").value;
            if (taskInput.trim() !== "") {
              let li = document.createElement("li");
              li.textContent = taskInput;

              // Step 2: Remove the task when clicked
              li.addEventListener("click", function() {
                li.parentNode.removeChild(li);
              });

              document.getElementById("taskList").appendChild(li);
              document.getElementById("taskInput").value = ""; // Clear the input field
            }
          });
        </script>

      </body>
      </html>
      ```

#8 Events and Event Listeners

- #1 Introduction to Events – Events are actions or occurrences that happen in the browser, such as a user clicking a button, typing into a form field, or submitting a form. Common types of events include
  - click: Fires when an element is clicked.
  - input: Fires when the value of an <input> or <textarea> changes.
  - submit: Fires when a form is submitted.
- #2 Adding Event Listeners – To respond to events, we use event listeners. The addEventListener() method attaches a function to an event on a specified element.

  ```
  let button = document.getElementById("myButton");
  button.addEventListener("click", function()
  {
    console.log("Button clicked!");
  });
  ```

- #3 Form Validation and Handling User Inputs
  - Form validation ensures that user input follows specific rules (e.g., valid email format, minimum password length).
  - This can be done using JavaScript before the form is submitted.

    ```
    function validateForm() {
      let email = document.getElementById("email").value;
      let password = document.getElementById("password").value;

      if (!email.includes("@")) {
        alert("Please enter a valid email address.");
        return false;
      }

      if (password.length < 6) {
        alert("Password must be at least 6 characters long.");
        return false;
      }

      return true;
    }
    ```

- code

  ```
  <!DOCTYPE html>
  <html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Events and Form Validation</title>
  </head>
  <body>

    <h1>Event Handling and Form Validation</h1>

    <!-- Button for Event Listener Example -->
    <button id="myButton">Click Me!</button>
    <p id="buttonStatus"></p>

    <!-- Form for Validation -->
    <form id="registerForm" onsubmit="return validateForm()">
      <label for="email">Email:</label>
      <input type="email" id="email" required><br><br>

      <label for="password">Password (min 6 characters):</label>
      <input type="password" id="password" required><br><br>

      <button type="submit">Submit</button>
    </form>

    <script>
      // Adding an Event Listener to the Button
      let button = document.getElementById("myButton");
      button.addEventListener("click", function() {
        console.log("Button clicked!");
        document.getElementById("buttonStatus").textContent = "Button was clicked!";
      });

      // Form Validation
      function validateForm() {
        let email = document.getElementById("email").value;
        let password = document.getElementById("password").value;

        // Check if the email contains "@"
        if (!email.includes("@")) {
          alert("Please enter a valid email address.");
          return false; // Prevent form submission
        }

        // Check if the password is at least 6 characters long
        if (password.length < 6) {
          alert("Password must be at least 6 characters long.");
          return false; // Prevent form submission
        }

        // If all validations pass
        return true;
      }
    </script>

  </body>
  </html>
  ```

- Exercise
  - Button with Event Listener – Add an event listener to a button that changes the text of a paragraph when clicked.

    ```
    <!DOCTYPE html>
    <html lang="en">
    <head>
      <meta charset="UTF-8">
      <meta name="viewport" content="width=device-width, initial-scale=1.0">
      <title>Button Event Listener</title>
    </head>
    <body>

      <h1>Change the Text</h1>
      <p id="text">This is the original text.</p>
      <button id="changeTextButton">Click me!</button>

      <script>
        // Add event listener to the button
        document.getElementById("changeTextButton").addEventListener("click", function()
        {
          document.getElementById("text").textContent = "The text has been changed!";
        });
      </script>

    </body>
    </html>
    ```

  - Form Validation – Create a form that validates user input (e.g., email format, minimum password length) before submission.

    ```
    <!DOCTYPE html>
    <html lang="en">
    <head>
      <meta charset="UTF-8">
      <meta name="viewport" content="width=device-width, initial-scale=1.0">
      <title>Form Validation</title>
    </head>
    <body>

      <h1>Register Form</h1>
      <form id="registerForm">
        <label for="email">Email:</label>
        <input type="email" id="email" required><br><br>

        <label for="password">Password (min 6 characters):</label>
        <input type="password" id="password" required><br><br>

        <button type="submit">Register</button>
      </form>

      <script>
        // Add form validation on submit
        document.getElementById("registerForm").addEventListener("submit", function(event) {
          let email = document.getElementById("email").value;
          let password = document.getElementById("password").value;

          if (!email.includes("@")) {
            alert("Please enter a valid email address.");
            event.preventDefault(); // Prevent form submission
          } else if (password.length < 6) {
            alert("Password must be at least 6 characters long.");
            event.preventDefault(); // Prevent form submission
          }
        });
      </script>

    </body>
    </html>
    ```

Presented with xmind